

Comparison of Fraud Detection Algorithms

Jan Bessler
d.velop AG
jan.bessler04@gmail.com

Simon Krempin
d.velop AG
simon.krempin@gmail.com

Abstract—Online transactions face an increasing amount of fraud, driven by invalid credit cards. The process of detecting such fraudulent transactions is a well researched area but yet difficult to navigate for novice researchers. To provide a initial introduction into the complex field, three methods are discussed and the most potent, *gradient boosting*, is then used to make a prediction about a test set.

I. EINFÜHRUNG

Mit der großen Beliebtheit von Online-Marktplätzen gehen neben vielen Vorteilen auch eine Vielzahl an Betrugsmaschinen einher. Eine besonders relevante ist die Verwendung von gefälschten Kreditkarten. Eine Vielzahl von Algorithmen wurde bereits für die Lösung des Problems entwickelt. In dieser Arbeit werden drei Algorithmen theoretisch erforscht, gegenübergestellt und ein ausgewählter Algorithmus an einem zur Verfügung gestellten Testdatensatz ausprobiert.

II. THEORETISCHER RAHMEN

Die Algorithmen für die Erkennung belaufen sich auf die drei folgenden:

Random Forest. Sie sind eine Sammlung aus einer Vielzahl von einzelnen *Decision Tree*'s. Jeder *Tree* ermittelt eigenständig die Vorhersage, über die für die finale Vorhersage ein Durchschnitt gebildet oder ein Mehrheitsvotum gehalten wird. [1]

Gradient Boosting. Gradient Boosting liefert einen ähnlichen Ansatz zu *Random Forest*'s. Die größte Unterscheidung liegt in einer sequenziellen Verarbeitung der Bäume, wodurch ein Nachfolger Fehler des Vorgängers durch erweiterte Informationen wie Fehlerquoten korrigieren kann. [2]

Support Vector Maschine (SVM). SVM hingegen basieren auf einer mathematischen Modellierung der Problemdomäne. Dabei wird versucht eine Trennlinie zwischen die unterschiedlichen Eingabeparameter zu zeichnen, anhand welcher neue Elemente kategorisiert werden können. Da die Eingabeparameter auf unterschiedlichen Skalen agieren, werden in einem Vorverarbeitungsschritt die Zahlen aneinander angepasst. [3]

Die Algorithmen werden durch eine Vielzahl an Parametern justiert, wodurch primär zwei Effekte entstehen, *Overfitting* und *Underfitting*. Dabei beschreibt *Overfitting* eine zu starke Annäherung an den Trainingsdatensatz. Daraus folgen gute Ergebnisse während des Trainings, aber schlechte Leistungen mit Echtdaten. Dem gegenüberstehend performt *Underfitting* ebenfalls während des Trainings schlecht, was auf einer zu einfachen Annahme der Problemdomäne fußt. Inwiefern die

einzelnen Parameter für ein *Overfitting* oder *Underfitting* sorgen, wird in Abschnitt IV diskutiert.

III. METHODIK

Für die Evaluation wird ein Datensatz bestehend aus 30.000 Datenpunkten herangezogen. Dieser folgt einer ungleichen Verteilung, wie für Kreditkartenbetrug üblich, sodass lediglich 1746 Datenpunkte (5.87%) einen Betrug darstellen. Für die korrekte Addressierung des stark unausgeglichene Datensatzes werden die Scores anhand des Macro-F1-Scores (mF1), der eine repräsentative Klassengewichtung widerspiegelt, verglichen. Während des Trainings wird eine Standard Test-split Methodik verfolgt, die den Datensatz in 80-20-Abschnitte einteilt. Dabei werden 80% für den Trainingsschritt verwendet und 20% für die anschließende Validierung. Während der Erforschung der Parameter wurde ein Cross-Fold Ansatz gewählt um die komplette Breite der Testdaten ausnutzen zu können.

Die Testdaten beinhalten unter anderem Artikelnummern in Form von den Spalten ANUMMER_01 bis ANUMMER_10. Eine Repräsentation der Artikel über eine Liste ist dahingehend ungeeignet, da zwei kritische Probleme damit einhergehen. Erstens sind die Bestellungen damit auf 10 begrenzt und Warenkörbe ab 11 Elementen können nicht akkurat dargestellt werden. Dazu kommt, dass eine hohe Anzahl an NULL-Werten gespeichert werden muss. Ein Vorteil einer solchen Darstellung ist eine verbesserte Performance der Query, da kein Leistung auf die JOIN-Operation aufgewandt wird. Auf der anderen Seite ermöglicht eine abgewandte Darstellung, indem die Artikel in eine weitere 1-n Tabelle ausgelagert werden, eine flexiblere Handhabung der Artikel. Durch eine Erweiterung zu einer n-m Tabelle, lassen sich ebenfalls Metadaten im normalisierten Zustand hinzufügen, die relevant für die Verbesserung der Vorhersage sein könnten. Für die weitere Bearbeitung der Aufgabe werden die Artikelspalten ausgeblendet. Des Weiteren werden alle Datumsspalten (B_GEBDATUM, Z_CARD_VALID, TIME_BEST, DATUM_LBEST) in die Einzelteile zerlegt, wie Monat und Jahr für die Kreditkarte. Die Kategorien (Z_METHODE, Z_CARD_ART, TAG_BEST, Z_LAST_NAME) werden über One-Hot-Encoding in valide Eingabeparameter umgewandelt, indem für jede eindeutige Kategorie eine weitere Spalte erstellt wird.

Für jeden Algorithmus werden die relevanten Parameter berücksichtigt und untereinander verglichen, um die optimale Performance pro Algorithmus zu berücksichtigen. Dazu werden die Parameter isoliert betrachtet und schrittweise skaliert, sodass eine Performancekurve entsteht. Um Interaktionseffekte zu berücksichtigen, wird auf empirische Erkenntnisse

zurückgegriffen, wodurch in diesen Fällen die Variablen auch zusammen betrachtet werden.

IV. ERGEBNISSE

A. Random Forest

Der einzige relevante Einflussfaktor für einen Random Forest ist die Anzahl der Bäume. Dabei stellt sich aber heraus, dass entgegen der Erwartung eine erhöhte Anzahl an Bäumen direkt für eine schlechtere Performance sorgt. Abb. 1 zeigt, dass bei einhundert Bäumen der mF1 bei 0,49 und bei einem Baum bei 0,53 liegt. Diese Diskrepanz fußt primär in einer falschen Spezialisierung, sodass keine Transaktion als Betrug eingestuft wird. Die null richtigen Ergebnisse in der Kategorie Betrug zahlen deshalb maßgeblich auf den schlechten Wert ein.

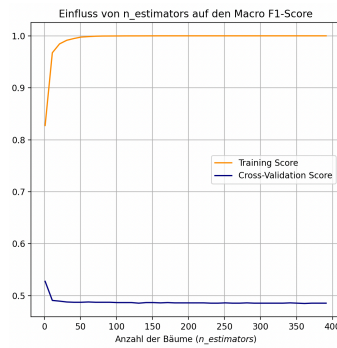


Fig. 1. Random Forest, mF1 Kurve für die Baumanzahl

B. Support Vector Maschine

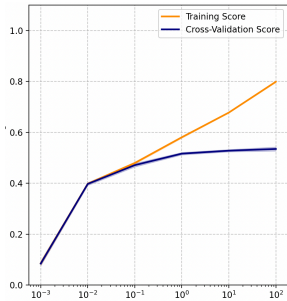


Fig. 2. SVM mF1 Kurve für C

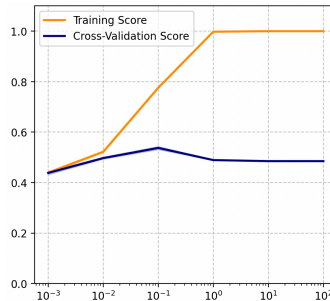


Fig. 3. SVM mF1 Kurve für Gamma

Abb. 2 zeigt, dass C einen stagnierenden Grenznutzen aufweist und sein Maximum bei einem Wert von 1000 findet. Anders ist die Kurve des Gamma-Wertes aus Abb. 3 zu bewerten, die einen lokalen Maximum bei 0,1 bildet.

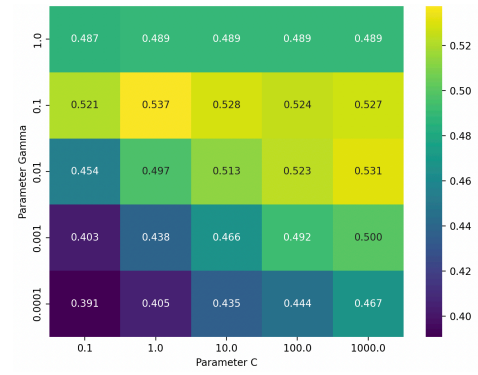


Fig. 4. SVM Matrix der Werte Gamma und C

Eine Betrachtung unter Berücksichtigung der Wechselwirkung der beiden Parameter wie in Abb. 4 bestätigt die Werte 0,1 für Gamma und 1000 für C. Der höchste mF1 mit 0,537 findet sich jedoch bei Gamma 0,1 und C 1, was eine höhere Relevanz von Gamma aufzeigt.

C. Gradient Boosting

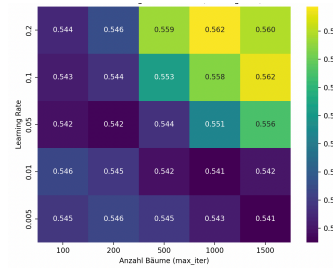


Fig. 5. Matrix Learnrate und Baumanzahl bei einfachen Bäumen

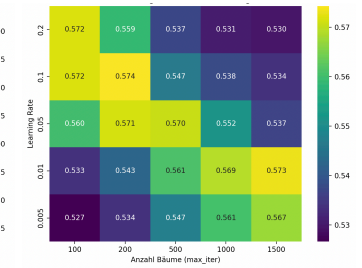


Fig. 6. Matrix Learnrate und Baumanzahl bei komplexen Bäumen

Die beiden Darstellungen Abb. 5 und Abb. 6 zeigen sowohl die Auswirkung der Lernrate im Zusammenspiel der Baumanzahl, als auch der Komplexität der Bäume. Daraus wird deutlich, dass komplexere Bäume einen positiven Effekt auf den mF1 haben. Bei weiterer Betrachtung der Abb. 6 ist eine invertierte Wechselwirkung zwischen Lernrate und Anzahl erkennbar. Daraus schließt sich für einen verbesserten mF1: je höher die Lernrate, desto geringer die Anzahl der Bäume.

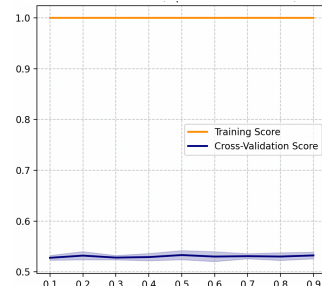


Fig. 7. Gradient Boosting, mF1 Kurve für L2-Regulierung

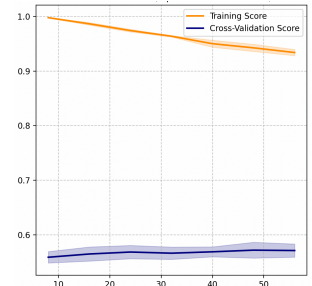


Fig. 8. Gradient Boosting, mF1

Die Kurve aus Abb. 7 zeigt einen marginalen Anstieg des mF1 mit einer Erhöhung der L2-Regulierung. Gleichmaßen steigt der mF1 mit einer Steigerung der Blätteranzahl und findet sein lokales Maximum bei 64 Blättern.

D. Gegenüberstellung

Schlussendlich entstehen für die drei Varianten die Tabelle Tabelle I. Während Random Forest und SVM einen nahezu ähnlichen Macro-F1-Score aufweisen, hebt sich die Performance von Gradient Boosting gegenüber der anderen deutlich (6%) ab.

Methode	Micro-F1-Score	Macro-F1-Score
Random Forest	0,89	0,53
SVM	0,90	0,54
Gradient Boosting	0,84	0,58

V. FAZIT

Aufgrundlage der aufgestellten Gegenüberstellung aus Abschnitt IV.D, wird Gradient Boosting für die finale Aufgabe verwendet. Der Datensatz umfasst 20.000 Datenpunkte. Gradient Boosting ermittelt 3/20 der Daten als Betrugsfälle. Das sind 15%, die über den 5.87% des Testdatensatzes liegen. Anhand dieser großen Diskrepanz lässt sich ohne die korrekten Ergebnisse beurteilen, dass Gradient Boosting zu inakkurat für die Aufgabe ist und für eine produktive Umgebung weitere Algorithmen erforscht werden sollten.

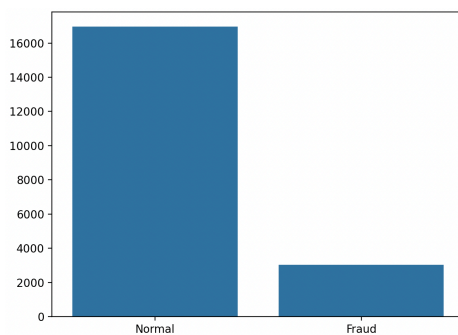


Fig. 9. Ergebnisse von Gradient Boosting auf Echtdaten

Dennoch konnte anhand der Algorithmen ein solider Einblick in das Datamining gegeben und durch die Erforschung der optimalen Parameter interessante Ergebnisse erzielt werden.

REFERENCES

- [1] L. Breiman, „Random forests“, *Machine learning*, Bd. 45, Nr. 1, S. 5–32, 2001.
- [2] J. H. Friedman, „1999 reitz lecture“, *Ann. Stat.*, Bd. 29, Nr. 5, S. 1189–1232, 2001.
- [3] C. Cortes und V. Vapnik, „Support-vector networks“, *Machine learning*, Bd. 20, Nr. 3, S. 273–297, 1995.